

DEVOPS | DOCKER | AWS EC2 | CLOUD DEPLOYMENT



Hospital Management System

Dockerized Deployment on AWS EC2

A 3-Tier MERN Application Containerized, Orchestrated, and Deployed From Scratch Using Real DevOps Practices

PROJECT TYPE	3-Tier MERN Stack Application (Forked & Dockerized)
EXAM	TrainWithShubham — DevOps Phase 1 Practical Exam
TECH STACK	React Node.js / Express MongoDB Docker AWS EC2
PREPARED BY	Sharad Verma

Connect on LinkedIn

 @sharadverma638

<https://github.com/sharadverma638/hospital-management-system-docker>

Original application forked from MKPTechnicals/Hospital-Management-System-MERN

Table of Contents

1. Project Summary
2. Objective
3. Original Repository & Tech Stack
4. System Architecture
5. Key Changes Made to the Original Code
6. Docker Setup & Optimisation
7. Git Hygiene & Commit History
8. AWS EC2 Deployment & Security
9. Application Walkthrough
10. Persistent Data — Test & Proof
11. Challenges Faced
12. What I Learned

1. Project Summary

This project takes an existing open-source 3-tier Hospital Management System, built on the MERN stack (MongoDB, Express, React, Node.js), and containerizes it end to end using Docker. The application is orchestrated with Docker Compose and deployed on an AWS EC2 instance. The original application had no Docker setup at all — this project adds the entire containerization and deployment layer without altering the app's core functionality.

This was completed as part of **TrainWithShubham — DevOps Phase 1 Practical Exam**, applying concepts of Linux, Git, Networking, System Design, and Docker on a real, unfamiliar codebase — rather than a toy project built from scratch.

2. Objective

- Take an existing 3-tier application (not built by me) and deploy it using real DevOps practices.
- Apply Docker best practices — multi-stage builds, minimal base images, and a non-root container user.
- Set up proper container-to-container networking using a custom Docker bridge network.
- Ensure the database's data persists independently of the container's lifecycle.
- Deploy the fully working application on AWS EC2 with correctly scoped security rules.
- Maintain clean Git hygiene with a meaningful, step-by-step commit history.

3. Original Repository & Tech Stack

The application was forked, as-is, from an existing open-source repository, confirming it was not already Dockerized:

Original repo: github.com/MKPTechnicals/Hospital-Management-System-MERN

Layer	Technology
Frontend	React (Create React App), Tailwind CSS, React Router
Backend	Node.js, Express.js, JWT Authentication, bcrypt
Database	MongoDB (via Mongoose)
Containerization	Docker (multi-stage builds), Docker Compose
Web Server	Nginx (serves the built React app in production)
Cloud Hosting	AWS EC2 (Ubuntu, Free Tier)

4. System Architecture

The application runs as three separate containers inside one EC2 instance, connected through a custom Docker bridge network. The frontend and backend ports are exposed publicly through the AWS Security Group, while MongoDB is kept completely internal — reachable only by the backend container. A named Docker volume keeps the database's data safe independent of the container's lifecycle.

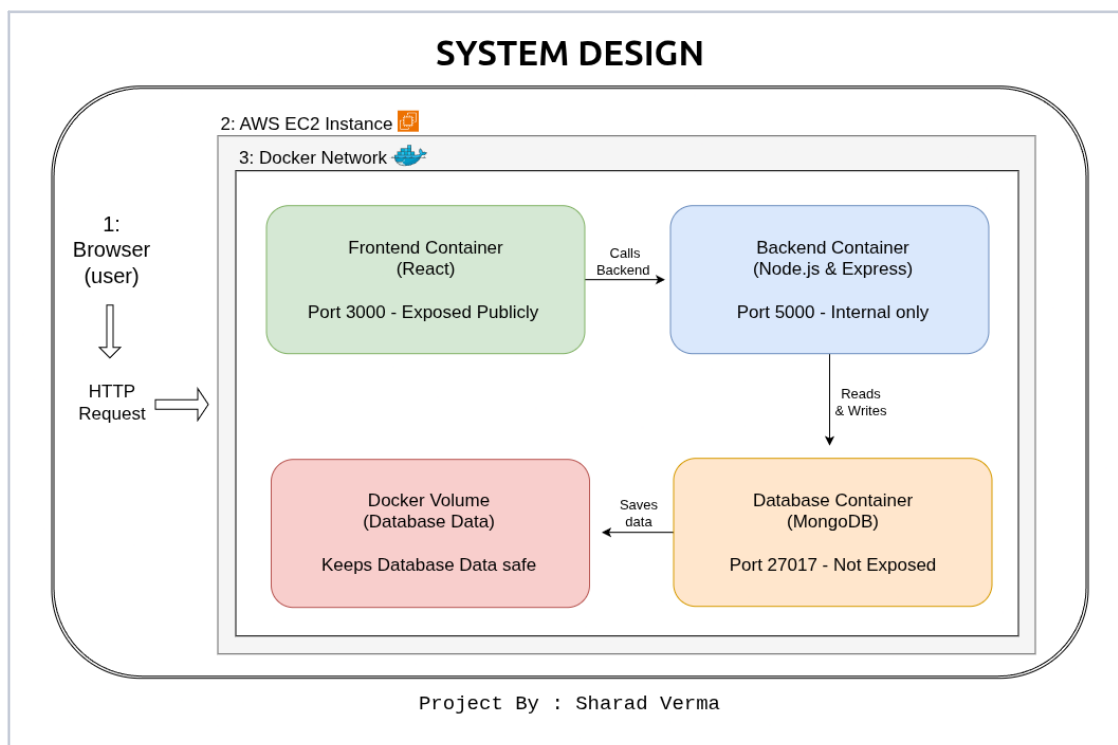


Figure 1 — System Design Diagram

Full 3-tier layout: browser to frontend, frontend to backend, backend to database, and the database's persistent volume. Ports and public/internal boundaries are labeled.

5. Key Changes Made to the Original Code

No core application logic, features, or database schema were changed. Only deployment-related configuration was added or modified:

5.1 Backend — Environment-Based Database Connection

server.js and createAdmin.js originally had a hardcoded placeholder for the MongoDB connection string. This was replaced with an environment variable so the database address is supplied securely at container runtime instead of being written into the source code:

```
mongoose.connect(process.env.MONGO_URI, {...})
```

5.2 Frontend — API URLs Fixed for Cloud Deployment

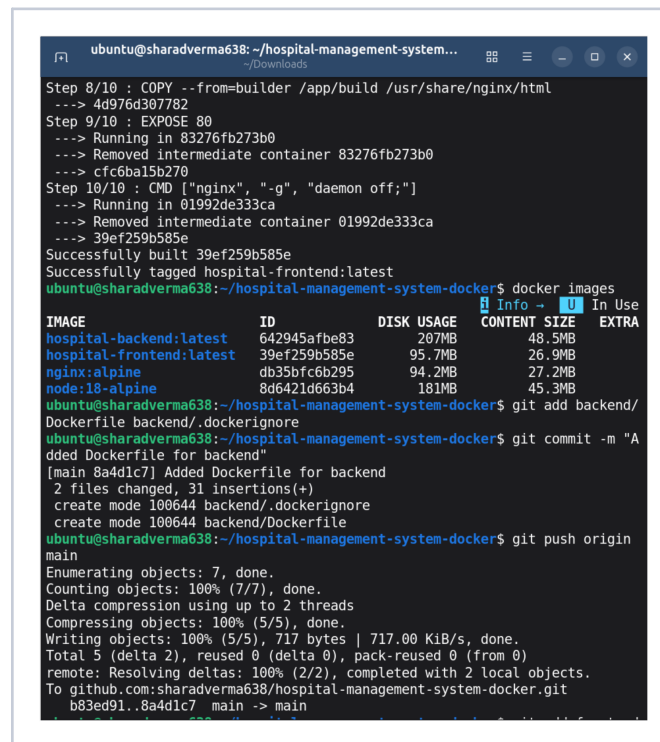
All ~20+ instances of hardcoded `http://localhost:5000` inside components (Login.js, SignUp.js, Patient.js, Doctors.js) were updated to use the EC2 instance's public IP. This was necessary because the React frontend runs as static files inside the user's own browser — it cannot resolve Docker service names or 'localhost' once deployed to the cloud.

5.3 New Files Added

- Dockerfile (backend) and Dockerfile (frontend) — none existed originally.
- .dockerignore (both services) — excludes node_modules, logs, and git files from the build.
- docker-compose.yml — orchestrates frontend, backend, and MongoDB together.
- update-ip.sh — automation script that updates the IP, rebuilds, and redeploys after every EC2 restart.
- .gitignore — standard Node/React ignore rules.

6. Docker Setup & Optimisation

- Multi-stage builds used for both services, so build tools and source code never ship in the final image.
- Alpine-based images (node:18-alpine, nginx:alpine) used to minimise final image size.
- Backend container runs as a dedicated non-root user for better security.
- Nginx serves the production frontend build instead of 'npm start' — lighter and production-appropriate.
- restart: unless-stopped set on all services for automatic recovery from crashes.



```

Step 8/10 : COPY --from=builder /app/build /usr/share/nginx/html
--> 4d976d307782
Step 9/10 : EXPOSE 80
--> Running in 83276fb273b0
--> Removed intermediate container 83276fb273b0
--> cfc6ba15b270
Step 10/10 : CMD ["nginx", "-g", "daemon off;"]
--> Running in 01992de333ca
--> Removed intermediate container 01992de333ca
--> 39ef259b585e
Successfully built 39ef259b585e
Successfully tagged hospital-frontend:latest
ubuntu@sharadverma638:~/hospital-management-system-docker$ docker images

```

IMAGE	ID	DISK USAGE	CONTENT SIZE	SIZE	EXTRA
hospital-backend:latest	642945afbe83	207MB		48.5MB	
hospital-frontend:latest	39ef259b585e	95.7MB		26.9MB	
nginx:alpine	db35bfc6b295	94.2MB		27.2MB	
node:18-alpine	8d6421d663b4	181MB		45.3MB	

```

ubuntu@sharadverma638:~/hospital-management-system-docker$ git add backend/
Dockerfile backend/.dockerignore
ubuntu@sharadverma638:~/hospital-management-system-docker$ git commit -m "A
dded Dockerfile for backend"
[main 8a4d1c7] Added Dockerfile for backend
2 files changed, 31 insertions(+)
create mode 100644 backend/.dockerignore
create mode 100644 backend/Dockerfile
ubuntu@sharadverma638:~/hospital-management-system-docker$ git push origin
main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 2 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 717 bytes | 717.00 KiB/s, done.
Total 5 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To github.com:sharadverma638/hospital-management-system-docker.git
b83ed91..8a4d1c7 main -> main

```

Figure 2 — Final Docker Image Sizes

Backend image: 207MB · Frontend image: 95.7MB — both kept lean via multi-stage builds and alpine base images.

7. Git Hygiene & Commit History

The repository was forked (not detached) to preserve proof that the original application was not already Dockerized. Work was committed in small, meaningful steps rather than one large commit — each Docker file, configuration change, and fix was committed separately as it was completed.

```
ubuntu@sharadverma638: ~/hospital-management-system-docker — ssh -i sharadverma638-key.pem ubuntu@ec2-54-166-179-110.compute-1.amazonaws.com
~/Downloads
Delta compression using up to 2 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (4/4), 325 bytes | 325.00 KiB/s, done.
Total 4 (delta 1), reused 1 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:sharadverma638/hospital-management-system-docker.git
   6419a01..7eac9f4  main -> main
ubuntu@sharadverma638:~/hospital-management-system-docker$ git log --oneline
7eac9f4 (HEAD -> main, origin/main, origin/HEAD) added image
6419a01 added docs
791ebe7 Create docs
fc3224a Revise README for Dockerized deployment details
45c12f0 Add automation script to update EC2 IP after restart
00727e9 Update frontend API calls to use EC2 public IP instead of localhost
e64ae40 Added docker-compose.yml to orchestrate frontend, backend, and MongoDB
b5db238 Use environment variable for MongoDB connection instead of hardcoded string
4e39a89 Use environment variable for MongoDB connection instead of hardcoded string
bd36221 Added Dockerfile for frontend
8a4d1c7 Added Dockerfile for backend
b83ed91 Added .gitignore
36b0890 Update login.js
cb7db37 Update README.md
e3076d4 Update README.md
572acd7 Update README.md
634c026 Update createAdmin.js
08e90d3 Add files via upload
7b958ce Update README.md
c4bdd95 Update README.md
bb0b167 Update README.md
aa5f941 Initial commit
ubuntu@sharadverma638:~/hospital-management-system-docker$ |
```

Figure 3 — Commit History

Step-by-step commits: Dockerfiles, environment variable fixes, docker-compose.yml, IP updates, and documentation — showing real incremental progress.

8. AWS EC2 Deployment & Security

The application was deployed on an AWS EC2 instance (Ubuntu, Free Tier). The Security Group was configured to expose only what is required: SSH for administration, and the frontend/backend ports for the application itself. MongoDB's port was never opened at the Security Group or Docker Compose level, keeping the database completely inaccessible from the internet.

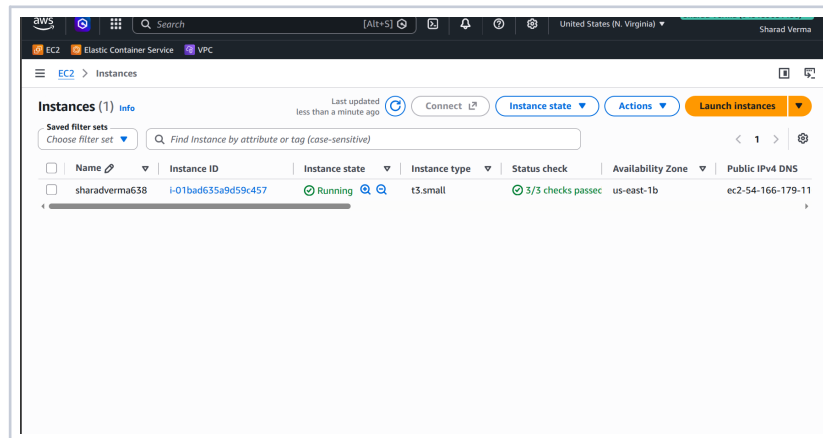


Figure 4 — EC2 Instance Running

AWS EC2 instance (t3.small, Ubuntu) in a running state with all status checks passed.

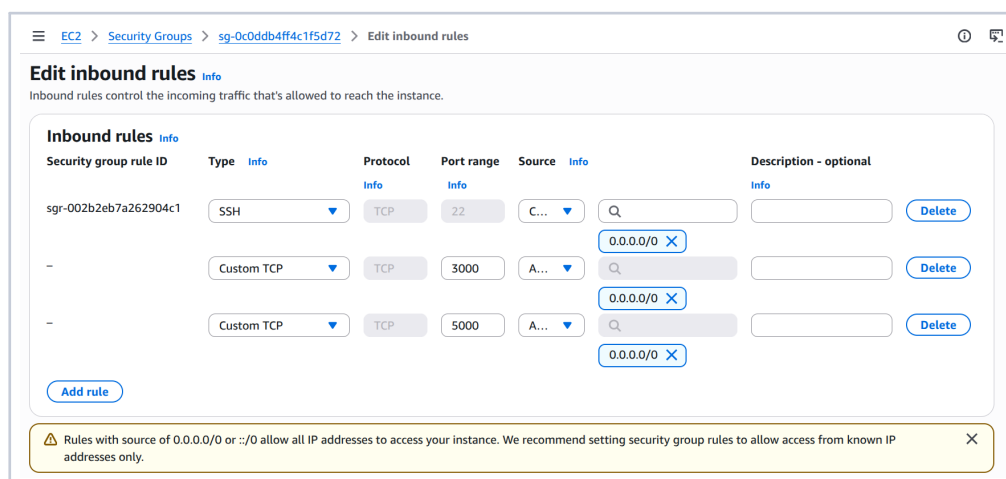
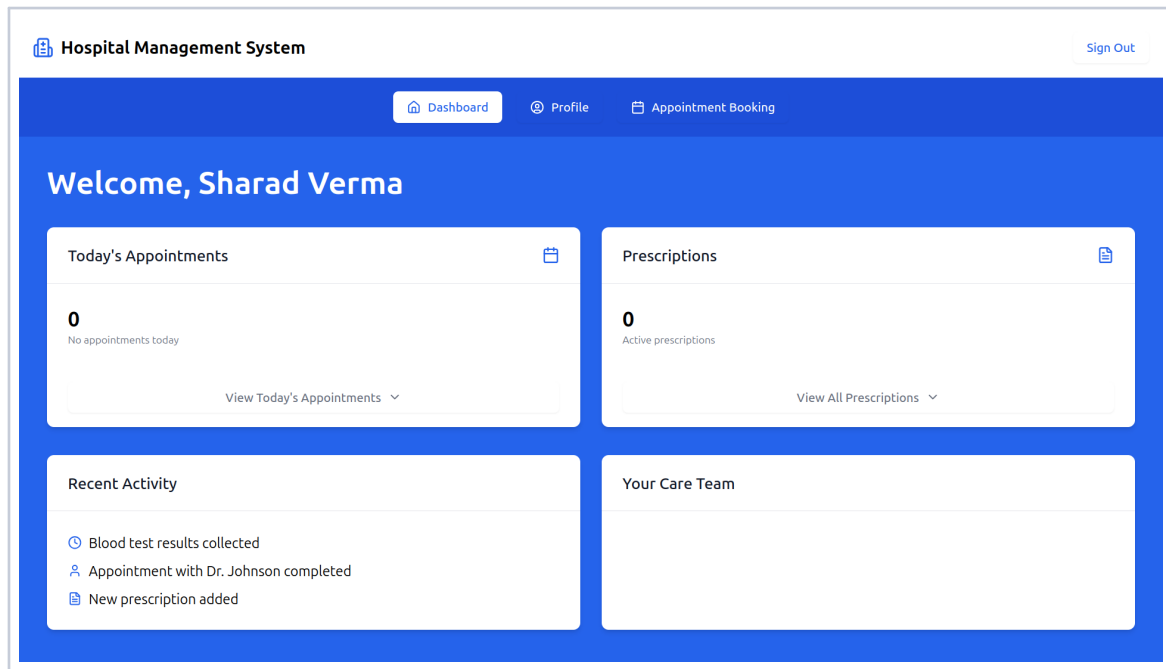


Figure 5 — Security Group Inbound Rules

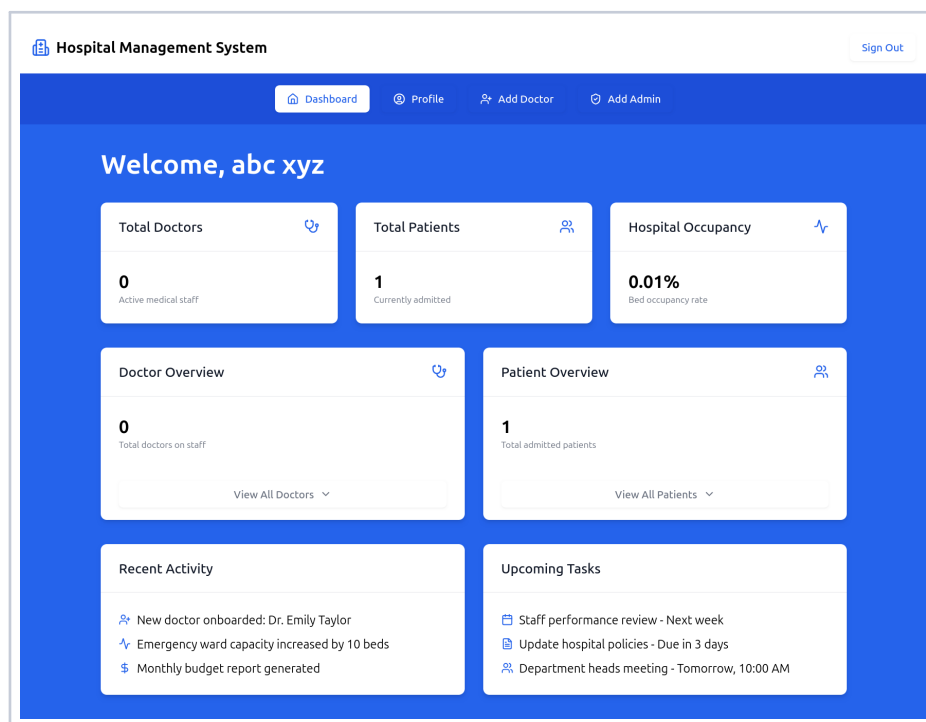
Only ports 22 (SSH), 3000 (frontend), and 5000 (backend) are open. Port 27017 (MongoDB) is intentionally absent.

9. Application Walkthrough

Once deployed, the application was tested end to end across all three user roles supported by the system — Patient, Admin, and Doctor — confirming that authentication, routing, and API communication all worked correctly through the deployed containers.

**Figure 6 — Patient Dashboard**

Patient view showing today's appointments, prescriptions, recent activity, and care team — served via the frontend container and live backend API calls.

**Figure 7 — Admin Dashboard**

Admin view showing hospital-wide stats — total doctors, patients, and occupancy — confirming the backend correctly aggregates data from MongoDB.

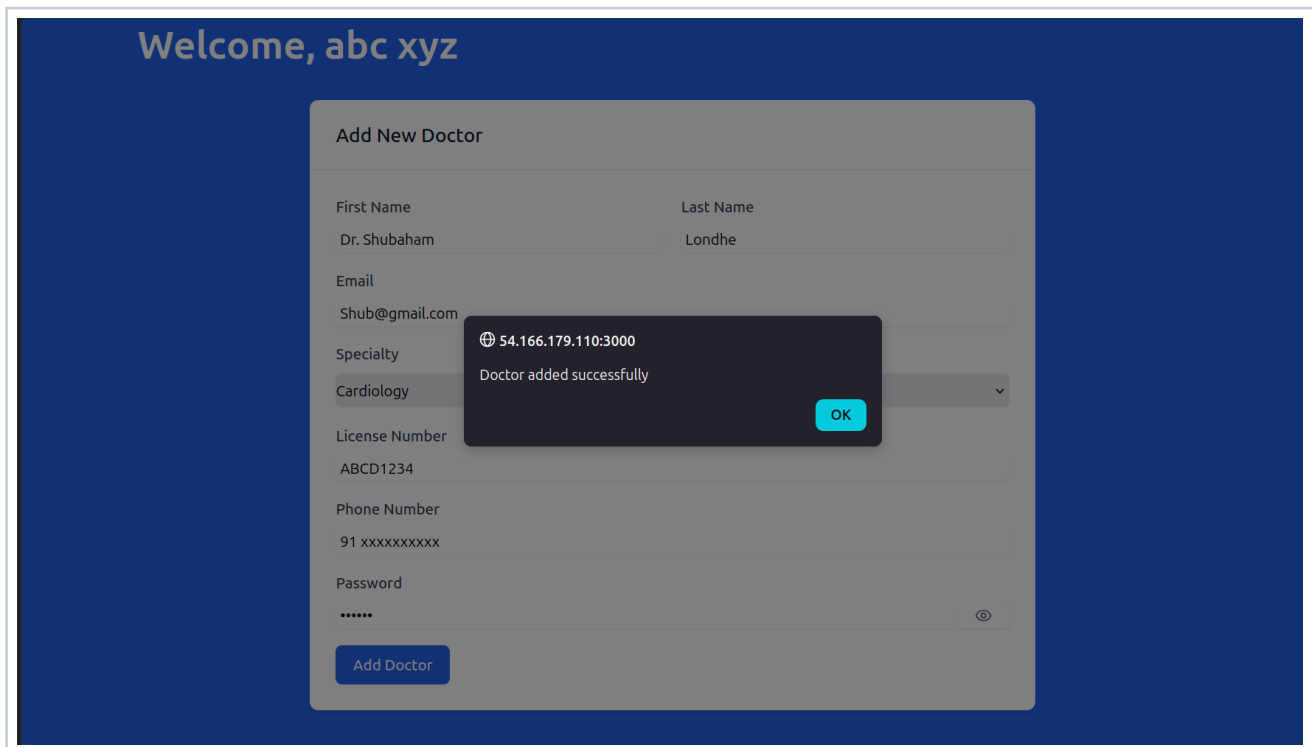


Figure 8 — Add Doctor (Admin Action)

A new doctor record being created from the Admin panel, with a success confirmation returned live from the backend API at the deployed EC2 public IP.

10. Persistent Data — Test & Proof

MongoDB stores its data at `/data/db` inside its container. A named Docker volume (`mongo-data`) is mounted to this path in `docker-compose.yml`, so the data lives independently of the container itself. To prove this works, the MongoDB container was deliberately stopped and removed — a full destroy, not just a restart — and then recreated from scratch using `docker-compose`.

```
mongosh mongodb://127.0.0.1:27017/hospital?directConnection=true&serverSelectionTimeoutMS=2000 — ssh -i sharadverma638-key.pem ubuntu@ec2-54-166...
~/Downloads
git: 'push' is not a git command. See 'git --help'.

The most similar command is
  push
ubuntu@sharadverma638:~/hospital-management-system-docker$ git push
Everything up-to-date
ubuntu@sharadverma638:~/hospital-management-system-docker$ docker exec -it hospital-mongo mongosh hospital
Current Mongosh Log ID: 6a94264b1f751091c09df8a2
Connecting to:      mongodb://127.0.0.1:27017/hospital?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.3
Using MongoDB:      6.0.28
Using Mongosh:      2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
  The server generated these startup warnings when booting
  2026-08-30T12:36:42.286+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
  2026-08-30T12:36:43.592+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
  2026-08-30T12:36:43.593+00:00: Soft rlimits for open file descriptors too low
  -----

hospital> db.patients.find().pretty()

hospital> db.users.find()
[
  {
    _id: ObjectId('6a94261032529d738f7aab0b'),
    firstName: 'Sharad',
    lastName: 'Verma',
    email: 'sv638@gmail.com',
    password: '$2b$10$scr0re9ABhNFI.VYCW7hLa.e3Xcrv3q0YfGgaF1mAz.NlujgRj2P7S',
    role: 'patient',
    _v: 0
  }
]
```

Figure 9 — Data Before the Test

Existing user data queried directly inside the running MongoDB container, before it is destroyed.

```

mongosh mongodb://127.0.0.1:27017/hospital?directConnection=true&serverSelectionTimeoutMS=2000 — ssh -i sharadverma638-key.pem ubuntu@ec2-54-166...
~/Downloads
[+] remove 1/1
✓ Container hospital-mongo Removed
ubuntu@sharadverma638:~/hospital-management-system-docker$ docker ps -a
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
8316acc064ac   hospital-management-system-docker-frontent  "/docker-entrypoint...."  13 minutes ago Up 13 minutes  0.0.0.0:3000->80/tcp, [::]:3000->80/tcp
060ae020a16e   hospital-management-system-docker-backend  "docker-entrypoint.s..."  13 minutes ago Up 13 minutes  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
ubuntu@sharadverma638:~/hospital-management-system-docker$ docker-compose up -d mongo
[+] up 1/1
✓ Container hospital-mongo Started
ubuntu@sharadverma638:~/hospital-management-system-docker$ docker exec -it hospital-mongo mongosh hospital
Current Mongosh Log ID: 6a9427101fa51ef5269df8a2
Connecting to:  mongodb://127.0.0.1:27017/hospital?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.3
Using MongoDB:  6.0.28
Using Mongosh:  2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

-----
The server generated these startup warnings when booting
2026-08-30T12:50:15.841+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
2026-08-30T12:50:16.825+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2026-08-30T12:50:16.825+00:00: Soft rlimits for open file descriptors too low
-----

hospital> db.users.find()
[
  {
    id: ObjectId('6a94261032529d738f7aab0b'),
    firstName: 'Sharad',
    lastName: 'Verma',
    email: 'sv638@gmail.com',
    password: '$2b$10$cr0re9ABhNFI.VYCW7hLa.e3Xcrv3q0YfGgaF1mAz.NlujgRj2P7S',
    role: 'patient',
    v: 0
  }
]

```

Figure 10 — Data After Container Recreation

The MongoDB container was removed and recreated via `docker-compose up -d mongo`. The exact same user record is still present — confirming the volume correctly persisted the data.

Result: Persistence confirmed. Data survives even when the database container is fully destroyed and rebuilt — proving the volume, not the container, is what actually holds the data.

11. Challenges Faced

1. Hardcoded 'localhost' URLs in the frontend

Since the React app runs in the user's own browser, not inside Docker, it could not resolve 'localhost' or Docker service names once deployed. Fixed by updating all API calls to the EC2 public IP and rebuilding the frontend image.

2. Docker Compose missing on EC2

A Snap-installed Docker conflicted with installing the Compose plugin via apt. Resolved by removing the Snap version and installing the standalone docker-compose binary directly.

3. Stale frontend image after code changes

Rebuilding the frontend manually with 'docker build' created a separate image that Docker Compose wasn't using, so old code kept running. Fixed by always rebuilding through Compose itself using `--no-cache`.

4. EC2 public IP changes on every restart

A fresh IP is assigned each time the instance restarts, breaking the frontend's backend URL. Solved by writing `update-ip.sh`, an automation script that updates the IP, rebuilds, and redeploys everything in one command.

5. Hardcoded database connection string

The original placeholder connection string was replaced with an environment variable (`MONGO_URI`), passed in securely through `docker-compose.yml` at runtime.

12. What I Learned

- The real difference between container-to-container communication (via Docker network/service names) and browser-to-server communication, which always needs a real, public address.
 - Why database ports should never be exposed publicly — and how to enforce that at both the Security Group and Docker Compose level.
 - How Docker volumes decouple data from a container's lifecycle, making persistence genuinely testable.
 - Practical AWS EC2 deployment: Security Groups, dynamic public IPs, and the operational quirks that come with running real infrastructure, not just following a tutorial.
 - How much of real DevOps work is debugging environment-specific issues rather than writing new code.
-

Repository: github.com/sharadverma638/hospital-management-system-docker

LinkedIn: linkedin.com/in/sharadverma638

Prepared for: TrainWithShubham — DevOps Phase 1 Practical Exam